

Model-Cascade Blended Cost, Worked Out by Hand

Why a four-tier cascade bills \$0.022 a query when the premium model alone bills \$0.08 — every tier weighted, for one refund-resolver traffic mix.

What this document does. It takes the one production traffic mix from Module 4.3 — the *refund-resolver cascade* that tries Haiku first and escalates only on low confidence — fixes the share of traffic that terminates at each of its four tiers (haiku, sonnet, opus, human), fixes the per-request cost at each tier, and then computes the single *blended* cost per query from first principles. Nothing is hand-waved. Each number below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module quotes a blended p50 of \$0.022 against an all-premium \$0.18; we rebuild the \$0.022 figure tier-by-tier and show exactly where the savings come from.

Where this sits. This is **Topic 1 of Module 4.3** — the headline number behind the cascade ladder (Haiku 62% → human 4%). Its companions: Module 3.4 (which *measured* the per-trajectory cost we are now blending) and Module 4.3 Topic 2 (batch discounts and circuit breakers — two more levers stacked on top of the cascade derived here).

Concepts covered, in order: the cascade as a probability-weighted mixture · tier probabilities p_i and per-request costs c_i · blended cost $\sum_i p_i c_i$ · the all-premium baseline · savings as $1 - \text{blended/premium}$ · the tier split as an estimate with standard error $\sqrt{p(1-p)/n}$ · how blended cost moves as the easy-tier share varies.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one traffic mix, small enough to count by hand

A model cascade is a routing rule. Each incoming request starts at the cheapest tier; if that tier answers with high enough confidence the request *terminates* there, otherwise it is escalated to the next tier up. Run this over a day of traffic and each request lands at exactly one tier. So the cascade partitions traffic into tiers, and the cost of a tier is paid only by the share of requests that stop there. The whole document is the consequence of one fact: a request costs whatever the tier it *terminated at* costs — nothing more.

The quantities. We give every tier a termination probability p_i (the fraction of traffic that stops at tier i) and a per-request cost c_i (what one query billed at that tier costs). These are the module's own production figures, rounded so the sums are followable on paper. The mechanics are identical at any numbers — only the constants change.

Tier	What it does	p_i	c_i
1 haiku / small	cache + simple rule, < 1 LLM call	0.62	\$0.002
2 sonnet / mid	1–2 tool calls + final answer	0.22	\$0.018
3 opus / large	full ReAct loop, escalated	0.12	\$0.080
4 human / escalate	complex / sensitive, handoff	0.04	\$0.180
Total probability		1.00	

Notation. p_i is the probability a random request terminates at tier i ; the p_i sum to 1 because every request stops somewhere. c_i is the cost of one request at tier i . The *blended* cost is the expected cost of a random request. We carry three decimals of a cent throughout.

Intuition

A cascade works because difficulty is *cheap to detect but expensive to handle*. A 50 ms classifier sorts traffic so that the easy 62% never touches the premium model. You are not paying the premium price on every query — you are paying it only on the 4% that genuinely needs a human. The blended bill is the premium price *diluted* by how rarely you reach for it.

2 Step 1 — the blended cost is a weighted average

The cost of a random request is a random variable: it equals c_i with probability p_i . Its expected value — the *blended cost* — is the probability-weighted sum

$$\text{blended} = \sum_i p_i c_i = p_1 c_1 + p_2 c_2 + p_3 c_3 + p_4 c_4. \quad (1)$$

This is just the definition of an expectation over the four tiers. Each term $p_i c_i$ is the *contribution* of tier i to the average bill: how often you reach it, times what it costs when you do. Working our four tiers:

Tier	Name	p_i	c_i	$p_i c_i$
1	haiku / small	0.62	\$0.002	\$0.00124
2	sonnet / mid	0.22	\$0.018	\$0.00396
3	opus / large	0.12	\$0.080	\$0.00960
4	human / escalate	0.04	\$0.180	\$0.00720
blended = $\sum_i p_i c_i$				\$0.02200

Sample check, tier 3: $0.12 \times \$0.080 = \0.0096 . ✓ Summing the four contributions: $0.00124 + 0.00396 + 0.00960 + 0.00720 = \0.02200 . ✓ The blended cost lands at **\$0.022** — exactly the module’s quoted p50.

Two things to notice already. First, the cheapest tier carries the *most* traffic but the *least* cost: tier 1 is 62% of requests yet only \$0.00124 of the bill. Second, the contributions are remarkably even — tiers 3 and 4 together are only 16% of traffic but more than three-quarters of the cost. **The bill is dominated by the rare, expensive tail**, not by the common cheap case.

Intuition

The blended cost is not the cost of an “average” query — there is no average query, only easy ones and hard ones. It is the cost you would pay *per query, on average*, if you ran the cascade over a large, representative day of traffic. Move the mix and the blend moves; the formula $\sum p_i c_i$ never changes.

3 Step 2 — the all-premium baseline and the savings

What does the cascade buy you? Compare it to the naive policy: send *every* request to the opus tier, since opus can handle anything tiers 1–3 can. Then every query costs $c_3 = \$0.080$, so the all-premium blended cost is just

$$\text{premium} = c_3 = \$0.080.$$

The cascade’s blended cost is \$0.022. The ratio is

$$\frac{\text{premium}}{\text{blended}} = \frac{0.080}{0.022} = 3.636\dots \approx 3.6 \times . \quad (2)$$

The cascade is $3.6\times$ **cheaper** than running opus on everything. Stated as a saving,

$$\text{savings} = 1 - \frac{\text{blended}}{\text{premium}} = 1 - \frac{0.022}{0.080} = 0.725 = 72.5\% \quad (3)$$

Check: $0.022/0.080 = 0.275$, and $1 - 0.275 = 0.725$. ✓ You keep 27.5¢ on the dollar; the cascade gives back the other 72.5¢.

Mini-example — this operation in isolation

The savings come entirely from *not* escalating. Of the dollar you would have spent at opus, the cascade only spends 0.275 of it, because $62\% + 22\% = 84\%$ of traffic stops at a tier far cheaper than opus. If instead every request *did* need opus, the cascade would collapse to the premium baseline ($p_3 \rightarrow 1$, blended $\rightarrow \$0.080$, savings $\rightarrow 0$). The cascade is worth exactly as much as your traffic is easy.

Watch out

The all-premium baseline here uses the opus per-request cost $c_3 = \$0.080$, not the human-handoff cost $c_4 = \$0.180$. “All-premium” means *the most capable model you would actually route to*, run on every query — not the cost of escalating everything to a human. Measure your baseline against the policy you are really replacing. If your fallback is human review, the relevant all-handoff baseline is \$0.180 and the cascade is $\$0.180/\$0.022 \approx 8.2\times$ cheaper still — a different, larger claim. Be explicit about which one you are quoting.

4 Step 3 — the tier split is itself an estimate

The probabilities p_i are not handed down from on high — you *measure* them by watching which tier each request terminates at. So each p_i is a sample proportion, and it carries sampling error. For a proportion p estimated from n requests, the standard error is

$$\text{SE}(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}. \quad (4)$$

Take the easy-tier share $p_1 = 0.62$. Then $p(1-p) = 0.62 \times 0.38 = 0.2356$, and:

n requests	$\text{SE}(\hat{p}_1)$	rough 95% band on p_1
100	0.0485	0.62 ± 0.10
1,000	0.0153	0.62 ± 0.03
10,000	0.0049	0.62 ± 0.01

Check, $n = 1000$: $\sqrt{0.2356/1000} = \sqrt{0.0002356} = 0.01535$. ✓ A day of a thousand requests pins the easy-tier share to about ± 3 points; you need ten thousand to get it inside ± 1 . Because the blended cost is linear in the p_i , that uncertainty flows straight through: a ± 0.03 wobble in p_1 (at the cheap end) moves the blend by only $\pm 0.03 \times (c_3 - c_1)$ or so — small, but worth a confidence band on any blended-cost figure you report.

5 Step 4 — how the blend moves with the easy-tier share

The single most important driver of the bill is p_1 : the fraction of traffic the cheapest tier can absorb. Hold the costs fixed and let p_1 vary, redistributing the remaining mass across tiers 2–4 in their original proportions (0.22 : 0.12 : 0.04). The blend responds sharply:

easy share p_1	p_2	p_3	p_4	blended
0.40	0.348	0.190	0.063	\$0.0336
0.50	0.290	0.158	0.053	\$0.0283
0.62	0.220	0.120	0.040	\$0.0220
0.70	0.174	0.095	0.032	\$0.0178
0.80	0.116	0.063	0.021	\$0.0125

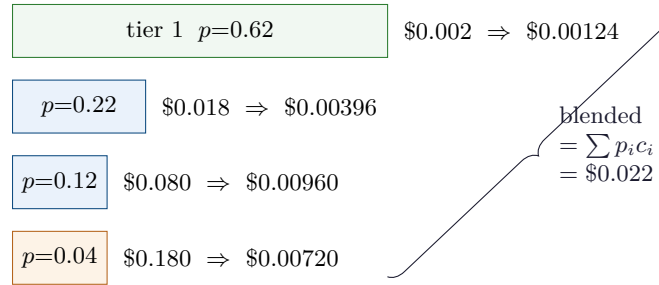
Sample check, $p_1 = 0.80$: the remaining 0.20 splits as $0.20 \times \frac{0.22}{0.38} = 0.1158$, $0.20 \times \frac{0.12}{0.38} = 0.0632$, $0.20 \times \frac{0.04}{0.38} = 0.0211$; blended = $0.80(0.002) + 0.1158(0.018) + 0.0632(0.080) + 0.0211(0.180) = \0.0125 . ✓

From $p_1 = 0.40$ to $p_1 = 0.80$ the blended cost roughly *halves* (\$0.034 $\rightarrow$ \$0.013). Every point of traffic you can push down to the cheap tier is a point of traffic that stops paying the expensive tail. This is why “raise the cheap-tier hit rate” is the highest-leverage knob in the whole cascade — and why the module warns that if tier 4 starts handling 30% instead of 4%, “your cascade is broken.”

Intuition

Confidence-based routing creates natural cost tiers because most real traffic is genuinely easy: FAQs, status lookups, duplicate-refund checks. The cascade’s job is to *recognise* easy and charge accordingly. The blended cost is a thermometer for that recognition — when it creeps up, either traffic got harder or the router got worse at spotting easy, and the two failure modes are told apart by watching the per-tier p_i , not the blend alone.

6 The cascade, drawn



Each bar's *width* is how often you reach that tier; the number on the right is that tier's contribution $p_i c_i$. The wide green bar (62% of traffic) is almost free; the two thin bars at the bottom (16% of traffic) carry most of the bill. The picture *is* Equation (1).

7 Cost cheat-sheet

Tier probabilities	$\sum_i p_i = 1$
Tier contribution	$p_i c_i$
Blended cost	$\text{blended} = \sum_i p_i c_i$
All-premium baseline	$\text{premium} = c_{\text{top}}$ (most-capable tier, on every query)
Cascade vs. premium	$\text{ratio} = \text{premium} / \text{blended}$
Savings	$1 - \text{blended} / \text{premium}$
Tier-split standard error	$\text{SE}(\hat{p}) = \sqrt{p(1-p)/n}$
Dominant driver	easy-tier share p_1 (each point down cuts the bill)

8 An honest word about these numbers

The per-request costs here (\$0.002, \$0.018, \$0.080, \$0.180) and the tier split (0.62, 0.22, 0.12, 0.04) are the module's illustrative production figures, picked round so the weighted sum is followable by hand. Your real tier costs depend on prompt sizes, tool counts, and model pricing; your real split depends on how easy your traffic is and how good your router is at spotting it — and that split *drifts*, which is why it is dashboarded, not assumed. What is *not* illustrative is the structure: a cascade's blended cost is the expectation $\sum_i p_i c_i$, full stop, and its savings over an all-premium policy are $1 - \text{blended} / \text{premium}$. Change the constants and the dollar figures move; the weighted-average shape does not. That shape is the thing to carry forward — it is why every cascade is monitored at the per-tier level, not just the blended bottom line.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the blended cost, the savings, the standard-error table, and the easy-tier sensitivity table; change the **probs**, **costs**, or **premium** to explore your own cascade's economics.

```
1 # cascade_cost.py -- reproduces every number in "Model-Cascade Blended Cost".
```

```

2 import math
3
4 names = ["haiku/small", "sonnet/mid", "opus/large", "human/escalate"]
5 probs = [0.62, 0.22, 0.12, 0.04] # share of traffic terminating at each tier
6 costs = [0.002, 0.018, 0.080, 0.180] # $ per request at each tier
7
8 assert abs(sum(probs) - 1.0) < 1e-9 # probabilities must sum to 1
9
10 # --- blended cost = sum_i p_i * c_i ---
11 blended = 0.0
12 for n, p, c in zip(names, probs, costs):
13     blended += p * c
14     print(f"{n:15s} p={p:.2f} c=${c:.3f} p*c=${p*c:.5f}")
15 print(f"blended = ${blended:.5f} (~ ${blended:.4f})")
16
17 # --- all-premium baseline = opus per-request cost, on every query ---
18 premium = costs[2] # $0.080
19 ratio = premium / blended
20 savings = 1 - blended / premium
21 print(f"premium = ${premium:.3f} ratio = {ratio:.3f}x savings = {savings*100:.1f}%")
22
23 # --- standard error of the easy-tier proportion ---
24 p1 = probs[0]
25 for n in (100, 1000, 10000):
26     se = math.sqrt(p1 * (1 - p1) / n)
27     print(f"n={n:6d} SE={se:.4f}")
28
29 # --- sensitivity: vary easy-tier share, split the rest proportionally ---
30 base = probs[1:]; bsum = sum(base) # 0.22, 0.12, 0.04 -> 0.38
31 for p_easy in (0.40, 0.50, 0.62, 0.70, 0.80):
32     rem = 1 - p_easy
33     pp = [p_easy] + [b / bsum * rem for b in base]
34     b = sum(pi * ci for pi, ci in zip(pp, costs))
35     print(f"p1={p_easy:.2f} blended=${b:.4f}")
36
37 # Expected output:
38 # haiku/small p=0.62 c=$0.002 p*c=$0.00124
39 # sonnet/mid p=0.22 c=$0.018 p*c=$0.00396
40 # opus/large p=0.12 c=$0.080 p*c=$0.00960
41 # human/escalate p=0.04 c=$0.180 p*c=$0.00720
42 # blended = $0.02200 (~ $0.0220)
43 # premium = $0.080 ratio = 3.636x savings = 72.5%
44 # n= 100 SE=0.0485
45 # n= 1000 SE=0.0153
46 # n= 10000 SE=0.0049
47 # p1=0.40 blended=$0.0336
48 # p1=0.50 blended=$0.0283
49 # p1=0.62 blended=$0.0220
50 # p1=0.70 blended=$0.0178
51 # p1=0.80 blended=$0.0125

```

— end of document —